stabilis — progress report

Frequency stability analysis engine. Milestones M0 and M1 complete.

`generated 2026-08-07 21:33 UTC · stabilis 0.0.1.dev0 · numpy 2.2.6 · python 3.10.12`
`commit 835e9ca (2 total) — WORKING TREE DIRTY · every figure below is measured at generation time`

What this is

Stable32 is the de-facto tool for frequency stability analysis. It is Windows-only, closed, GUI-driven, unscriptable and no longer developed. *stabilis* is an open-core engine for the same job, intended as the free foundation under a paid workbench that produces signed ISO-17025-style reports for calibration labs and oscillator manufacturers.

Two things distinguish it from a reimplementation. First, the Haar wavelet variance *is* the Allan variance, so classical AVAR is the K=1 case of one framework; higher-order Daubechies filters then buy drift immunity and convergence for steeper noise. Second, confidence intervals are derived from the estimator's own coefficient autocovariance, so they do not require identifying the noise type first — Stable32 looks it up in a table and has no answer when the noise is not one of its five power laws.

> **The commercial argument for validation.** ISO/IEC 17025:2017 clause 6.4.1 defines software as *equipment*, and 7.11.2 requires laboratory software to be validated before introduction. Its NOTE 2 exempts commercial software "in general use within its designed application range" — an exemption stabilis fails twice over, being new and being open-core, since a lab that patches it explicitly triggers re-validation. An accredited lab therefore cannot adopt this engine unless it ships validation evidence. That evidence is the product's moat, not its overhead.

Results: accuracy against certified values

Every classical variance is a sum of squares of integer linear combinations of the phase samples, divided by an integer. Both NBS14 reference sets are integer-derived, so on them the variance is an **exact rational**. It is computed independently in `fractions.Fraction`, with the deviation certified to 50 digits via `decimal.Decimal.sqrt()`. That gives certified values computed here, to far more digits than any published table, with no dependence on a transcription being correct.

Accuracy is reported as **LRE**, the log relative error, after the NIST Statistical Reference Datasets convention: it is the number of correct significant digits. float64 carries about 15.95, so these are at the representation limit.

Estimator	NBS14 9-point (min LRE)	NBS14 1000-point (min LRE)
adev	16.3	14.8
oadev	16.1	14.8
mdev	16.3	14.1
tdev	16.0	14.1
ohdev	16.2	14.5

A worked certified value

Overlapping Hadamard variance of the NBS14 nine-point set at AF 1, as an exact rational and as a 50-digit deviation:

exact variance	`10027 / 2`
certified deviation	`70.806073185850378720054399356481405765799283518283`
computed (float64)	`70.80607318585038`

> **The oracle earned its keep immediately.** The relayed NBS14 table gave `hdev(1) = 70.80608` while listing `ohdev(1) = 70.80607` — but these are the same estimator at AF 1, since there is no overlap to exploit. The certified value settles it at `70.806073185850378`, so the first was a transcription error. This is exactly why correctness rests on algebra rather than on published tables.

Results: analytic identities

These need no reference data and no oracle, which is why they carry the correctness argument. Every published ADEV table traces back to Stable32, so agreeing with one proves interoperability; agreeing with algebra proves correctness.

Identity	Claim	Result
linear frequency drift: avar = $D^2 \cdot \tau^2/2$	bit-exact	**PASS**
linear frequency drift: hvar = 0	bit-exact	**PASS**
constant frequency offset leaves avar unchanged	bit-exact	**PASS**
scale equivariance at 2^k: var → 64·var for c = 8	bit-exact	**PASS**
adev = oadev = mdev at AF 1	rel 1e-15	**PASS**
tdev = $\tau/\sqrt{3}$ · mdev	rel 1e-15	**PASS**
constant phase gives exactly zero	bit-exact	**PASS**

> **The most valuable single test.** For a pure linear frequency drift D, the phase is quadratic, so the second difference is the constant $D \cdot \tau^2$ and the third difference vanishes. Hence `avar = D²τ²/2` and `hvar = 0`, both exactly. One test validates the Allan normalization, the Hadamard normalization and the drift convention simultaneously — and it is bit-exact, not approximate.

Results: interoperability with Stable32

Labelled as interop, not correctness: these published values were produced by Stable32 itself. Seven significant figures as published, so agreement is assessed at rtol 1e-6.

Estimator	AF	computed	published	rel. diff
adev	1	2.9223188e-01	2.9223190e-01	7.5e-08
adev	10	9.9657361e-02	9.9657360e-02	6.3e-09
adev	100	3.8978043e-02	3.8978040e-02	8.5e-08
oadev	1	2.9223188e-01	2.9223190e-01	7.5e-08
oadev	10	9.1599534e-02	9.1599530e-02	4.6e-08
oadev	100	3.2413430e-02	3.2413430e-02	8.0e-09
mdev	1	2.9223188e-01	2.9223190e-01	7.5e-08
mdev	10	6.1723764e-02	6.1723760e-02	6.2e-08
mdev	100	2.1709209e-02	2.1709210e-02	4.0e-08
tdev	1	1.6872015e-01	1.6872020e-01	2.8e-07
tdev	10	3.5636232e-01	3.5636230e-01	4.7e-08
tdev	100	1.2533818e+00	1.2533820e+00	1.8e-07
ohdev	1	2.9438833e-01	2.9438830e-01	9.9e-08
ohdev	10	9.5810832e-02	9.5810830e-02	1.8e-08
ohdev	100	3.2376383e-02	3.2376380e-02	7.8e-08

Results: reference generator and numerics

NBS14 1000-point set ships as arithmetic, not a file

The set is defined by a published Lehmer recursion `k[i+1] = 16807·k[i] mod 2147483647` from seed `1234567890`, so it needs no data file and carries no licence question. Verified against the published `wriley.com/tst_suit.dat`: all 1000 values agreed to 5.55e-16 — one ULP, which is that file's own text rounding. **The generator is the more exact artifact of the two.**

k[0]	`1234567890`
k[1] (check value)	`395529916` PASS
vs published digits	`max dev 3.33e-16`

Phase ↔ frequency round-trip

Recovers the offset-removed phase to a few ULP. Not bit-exact: a cumulative sum of differences does not telescope exactly in floating point. The test bounds this at 8 ULP, a figure derived from the measurements below rather than tuned.

n	peak (s)	max error (s)	ULP
10	2.690e-12	1.010e-28	0.2
1000	5.009e-11	6.462e-27	1.0
100000	2.415e-10	1.292e-25	2.5

Power-law noise synthesis

Kasdin–Walter generation, log-log periodogram slope recovered over 1e-3 < f < 0.1, averaged over eight realizations.

α (requested)	slope (recovered)	|error|
2	+1.9734	0.0266
1	+0.9912	0.0088
0	+0.0043	0.0043
-1	-0.9943	0.0057
-2	-1.9955	0.0045

Design decisions worth recording

Scale factors live outside the coefficients

Every estimator returns the coefficient array it averaged, so one confidence-interval implementation can serve all of them. The obvious form is `variance = mean(c²)`, which requires dividing by √2 or √6. That was implemented and then reverted: it makes every coefficient irrational and destroys exactness where the mathematics is exact — the drift identity above failed by one bit. The contract is now `variance = scale · mean(c²)`. EDOF depends only on the scale-free correlation structure, so nothing is lost.

float64 phase is a genuine hazard at this precision

At an accumulated phase of 10^3–10^6 s the float64 spacing is 10^{-13}–10^{-10} s, which silently quantizes away exactly the picoseconds sub-nanosecond work cares about. Phase is therefore stored as offsets from an explicit epoch, and construction warns when the spacing approaches the smallest resolved step. The warning is recorded in provenance, not merely printed, so it survives into a report.

The classical estimators are not routed through the wavelet filter bank

The Haar-equals-Allan identity is the intellectual core of the library, but making it the *implementation* would put boundary handling and shift-equivariance on the critical path for no user-visible gain. Written directly, the identity becomes an independent check of two separate code paths instead of a tautology.

No pickle support, by design

`pickle.load` on a file arriving from a customer's test rig is arbitrary code execution. `.npy` covers the same need safely.

Current state

test suite	`154 passed in 2.03s` PASS
wall time	`2.3 s`
latest commit	`835e9ca — Add the classical estimators and an exact-rational oracle (M1)`

Three structural properties are enforced by tests rather than convention: the core computes with scipy and matplotlib absent; every theory page must declare its validation status and name a covering test; and built-in readers and estimators register through the same decorator a third-party plugin would use.

Built

- **M0** — immutable phase/frequency containers with the precision guard, provenance with a step log, the plugin registry, tau selection, Kasdin–Walter synthesis, delimited-text reading, theory pages, and a CLI.
- **M1** — `adev`, `oadev`, `mdev`, `tdev`, `ohdev` computed directly from phase, gap-aware, plus the exact-rational oracle and the Tier 0 identity suite.

Next

- **M2** — tapered η_1 and η_3 degrees of freedom, the pure-numpy inverse χ^2, confidence intervals, noise identification, and `stabilis validate` emitting a versioned evidence report. This is the v0.1 release point and the first genuinely saleable cut.
- **M3** — Greenhall EDOF for error-bar parity with Stable32, the MODWT wavelet layer, TOTVAR with its bias correction, power-law decomposition.
- **M4–M7** — model fitting against the χ^2 likelihood, instrument readers, realtime ingest and dynamic Allan variance, then the remaining parity tail and the report layer.

> **Known limitations.** No confidence intervals yet, so no uncertainty may be quoted from this version. Expected-value tables relayed from secondary sources are marked UNVERIFIED in the source pending transcription from the SP 1065 PDF; the certified oracle does not depend on them. Instrument readers are not yet validated against real hardware exports. TOTVAR, Theo1, MTIE and the wavelet layer are not implemented.